LS10 · Debugging-Labor

Auftrag und Abgabeprotokoll · Bibliotheksabrechnung · Einzelarbeit

Ausgangslage

Du übernimmst ein bestehendes Bibliotheksprogramm. Es lässt sich starten, liefert aber nicht immer das fachlich richtige Ergebnis. Du erhältst den unveränderten Startcode und einen Ist-Programmablaufplan (Ist-PAP), der diesen Startcode abbildet.

Ziel: Entdecke Fehler SELBST mit eigenen Tests und dem Visual-Studio-Debugger. Belege zuerst ihre Ursache, korrigiere dann gezielt den C#-Code und erstelle ERST DANACH selbst deinen Soll-PAP aus einer Kopie des Ist-PAP.

Verbindliche Fachregeln

- Ausleihtage sind ganze Zahlen ab 1. Bei 0 oder negativen Zahlen erscheint eine Fehlermeldung; es wird keine Gebühr ausgegeben.
- Bis und mit Tag 7 ist die Ausleihe kostenlos (CHF 0.00).
- Ab Tag 8 kostet jeder zusätzliche Tag über 7 CHF 0.50.
- Die Höchstgebühr ist CHF 10.00.
- Menüaufbau ○ 1 = Gebühr berechnen ○ 0 = beenden ○ andere ganze Zahlen = Hinweis und erneutes Menü.

Für den Pflichtauftrag kannst du ganzzahlige Konsoleneingaben voraussetzen. Texteingaben wie «abc» gehören nicht zur Fehlersuche.

Deine Ausgangsmaterialien

- Visual-Studio-Projekt «BibliothekDebug» mit «Program.cs» (nicht vorschnell verändern).
- «Ist_PAP-Bibliotheksabrechnung.drawio» beschreibt den fehlerhaften Ausgangszustand.
- Dieses Dokument dient zugleich als ausfüllbares Diagnose- und Abgabeprotokoll.

Wichtig: Anzahl, Position und Ursachen der Fehler sind NICHT vorgegeben.

Arbeitsauftrag | Reihenfolge einhalten

Bearbeite die Schritte in dieser Reihenfolge. Ein funktionierendes Programm allein ist noch kein vollständiger Debugging-Nachweis.

Zeit	Schritt	Nachweis
15 Min.	Mindestens sechs eigene Testfälle und einen Menütest entwerfen, Soll-Ergebnisse begründen.	Testplan VOR dem Korrigieren
20 Min.	Ursprünglichen Code mit Ist-PAP vergleichen, erste Vermutungen formulieren.	Zwei eigene Hypothesen
20 Min.	F9, F10 und Variablenansicht am Menüzweig üben.	Debugger-Einstiegsbeleg
55 Min.	Auffälligkeiten selbst finden; pro Ursache reproduzieren, Variable/Zweig beobachten, dann korrigieren und nachtesten.	Diagnosekarte je Ursache
35 Min.	Ist-PAP kopieren, selber entsprechend dem korrigierten Code zum Soll-PAP verändern.	Eigener editierbarer Soll-PAP
25 Min.	Regression aller Testfälle und Code-PAP-Abgleich auf mindestens zwei Wegen.	Regression + 2 Debugger-Bilder
10 Min.	Git-Commit, Reflexion und Abgabe vorbereiten.	Abgabe und Commit-ID

Debugger: Beobachtung statt Raten

- Erwartung vor dem Test notieren und das tatsächliche Ergebnis vergleichen.
- Hypothese formulieren und Breakpoint (F9) an einer begründeten Stelle setzen.
- Mit F10 schrittweise arbeiten, Variablenwerte und tatsächlichen Ja-/Nein-Weg beobachten.
- Ursache in eigenen Worten erklären; erst dann den Code gezielt ändern und nachtesten.

PAP: Notation und Formate

Notation und dessen graphischen Formen beachten. Rücksprünge und Programmende bleiben erkennbar.

Der fertige Soll-Programablaufplan wird als editierbare «.drawio» Datei abgegeben, ein Screenshot oder nur das PDF genügt nicht.

Unterstützung und Differenzierung

Einstiegshilfe: Verfolge zuerst einen bekannten Menüzweig mit F9/F10. Bitte bei Schwierigkeiten um Hinweise zur Diagnose, nicht um die korrigierte Zeile.

Optional nach dem Pflichtkern: drei zusätzliche Grenztests begründen, mehrere Berechnungen in einem Lauf untersuchen oder die Konsolentexte verbessern, ohne die Regeln zu verändern.

Verwende ausschliesslich bekannte Konzepte: Variablen, Rechnen, Ein-/Ausgabe, Verzweigungen und Schleifen. Keine eigenen Methoden, Klassen oder Arrays.

Arbeitsprotokoll | Vorbereitung

1 Eigener Testplan – VOR dem ersten Programmstart

Leite mindestens sechs Fälle selbst aus den Fachregeln ab: ungültige Dauer, kostenloser und kostenpflichtiger Fall, mindestens zwei relevante Grenzwerte. Berechne die Sollwerte vorab und begründe zwei deiner Fälle.

ID	Eingabe / Bedienfolge	Soll-Ergebnis	Begründung / geprüfte Regel
T1	Menü 1, dann Tage 0	Fehlermeldung, keine Gebühr	Tage müssen mindestens 1 sein. 0 ist ungültig.
T2	Menü 1, dann Tage -3	Fehlermeldung, keine Gebühr	Negative Tage sind ungültig.
T3	Menü 1, dann Tage 7	CHF 0.00	Bis und mit Tag 7 ist es kostenlos.
T4	Menü 1, dann Tage 8	CHF 0.50	Tag 8 ist 1 Tag über 7, also 1 × 0.50.
T5	Menü 1, dann Tage 12	CHF 2.50	12 − 7 = 5 Tage, 5 × 0.50 = 2.50.
T6	Menü 1, dann Tage 40	CHF 10.00	40 − 7 = 33 Tage, 33 × 0.50 = 16.50. Das ist über dem Maximum, also CHF 10.00.
Menü 5,	dann 1, dann 8, dann 0	«Bitte 0 oder 1 wählen.», Menü erneut, CHF 0.50, «Programm beendet.»	Andere Zahl gibt einen Hinweis und das Menü kommt wieder. 0 beendet.

Welcher selbst gewählte Grenzwert Besonders aussagekräftig ist der Wechsel von 7 zu 8 Tagen, weil genau dort die Gebühr von gratis zu kostenpflichtig wechselt. Wenn die Bedingung oder die Rechnung um einen Tag verschoben ist, sieht man das nur bei diesen beiden Werten. ist besonders aussagekräftig? Warum?

Stoppregel: Zu jeder entdeckten Ursache füllst du vor der Korrektur die Diagnosekarte auf der nächsten Seite aus.

---

2 Ist-PAP und Originalcode verstehen

Öffne den Ist-PAP und den noch unveränderten Original C#-Code. Ändere noch nichts.

Beschreibe den Weg vom Menü bis zu einer berechneten Gebühr:

Das Programm zeigt das Menü und liest auswahl ein. Bei 1 fragt es die Ausleihtage ab und prüft, ob die Dauer gültig ist. Bei gültiger Dauer berechnet es die zusätzlichen Tage und daraus die Gebühr. Danach prüft es die Höchstgebühr und gibt Gebühr aus. Dann geht es zurück zum Menü.

Wie wird entschieden, ob das Menü erneut angezeigt wird?

Die Variable weiter steuert die while-Schleife. Sie ist am Anfang true. Nur bei Auswahl 0 wird sie false, dann endet das Programm. Bei allen anderen Eingaben läuft die Schleife weiter und das Menü erscheint wieder.

Hypothese A: Welche Stelle würdest du zuerst genauer untersuchen? Warum?

Ich untersuche die Zeile if (tage >= 7), weil ich vermute, dass sie bei Tage = 7 falsch reagiert. Laut Regel ist Tag 7 noch gratis.

Hypothese B: Welche weitere Stelle würdest du untersuchen? Warum?

Ich untersuche die Zeile gebuehr = zusatztage * 0.75m;, weil ich vermute, dass der Preis pro Tag nicht zur Regel passt. Laut Regel müssten es CHF 0.50 sein.

3 Debugger-Einstieg am Menü

Setze einen Breakpoint vor der Menüauswertung und untersuche einen erwarteten Menüzweig mit F10.

Haltepunkt / Codezeile	Wert von «auswahl»	Ausgeführter Zweig und Begründung
if (auswahl == 0) in Program.cs	1	auswahl == 0 ist Nein, auswahl == 1 ist Ja. Es läuft der Zweig «Ausleihtage abfragen», weil ich im Menü 1 eingegeben habe.

Diagnosekarte | Für jede Ursache kopieren

Du bestimmst selbst, wie viele unterschiedliche Fehlerursachen vorhanden sind. Dupliziere diese Seite so oft wie nötig. Verschiedene Symptome desselben Fehlers gehören in eine einzige Karte.

Diagnosekarte 1

Was wird dokumentiert?	Dein Befund (VOR / NACH der Korrektur)
Meine Bezeichnung für die Ursache	Prüfung Ausleihdauer: 0 wird nicht abgelehnt
Test-ID und Eingabe	T1, Menü 1, Tage 0
Erwartetes Ergebnis (Soll)	Fehlermeldung «Ungültige Ausleihdauer.», keine Gebühr
Beobachtetes Ergebnis (Ist)	Zusätzliche Tage: 0, Gebühr: CHF 0.00
Vermutete Ursache / Hypothese	Die Bedingung tage < 0 in Zeile 25 lässt 0 durch.
Breakpoint (Datei / Stelle)	Program.cs, Zeile 25, if (tage < 0)
Variablenwerte und ausgeführter Zweig	tage = 0. Die Bedingung ist false, es läuft der else-Zweig (Zeile 30), die Gebühr wird berechnet.
Warum belegt dies die Ursache?	Laut Regel ist 0 ungültig. Der Debugger zeigt, dass bei tage = 0 die Fehlermeldung (Zeile 27) übersprungen wird, weil 0 nicht kleiner als 0 ist.
Gezielte Codekorrektur (erst jetzt!)	alt: if (tage < 0) neu: if (tage <= 0)
Direkter Nachtest: Soll / Ist	Soll: Fehlermeldung «Ungültige Ausleihdauer.» Ist: «Ungültige Ausleihdauer.», keine Gebühr → bestanden
PAP-Stelle, die dadurch angepasst werden muss	Raute «tage < 0?» wird zu «tage <= 0?»

Diagnosekarte 2

Was wird dokumentiert?	Dein Befund (VOR / NACH der Korrektur)
Meine Bezeichnung für die Ursache	Grenze gratis/kostenpflichtig um 1 Tag verschoben (Bedingung und Abzug)
Test-ID und Eingabe	T3: Menü 1, Tage 7 (zusätzlich T4: Menü 1, Tage 8)
Erwartetes Ergebnis (Soll)	T3: Zusätzliche Tage 0, CHF 0.00 / T4: Zusätzliche Tage 1, CHF 0.50
Beobachtetes Ergebnis (Ist)	T3: Zusätzliche Tage: 1, CHF 0.75 / T4: Zusätzliche Tage: 2, CHF 1.50
Vermutete Ursache / Hypothese	Hypothese A: if (tage >= 7) behandelt Tag 7 schon als kostenpflichtig; zusätzlich zählt tage - 6 einen Tag zu viel.
Breakpoint (Datei / Stelle)	Program.cs, Zeile 33, if (tage >= 7), danach mit F10 bis Zeile 37
Variablenwerte und ausgeführter Zweig	T3: tage = 7, Bedingung true → Ja-Zweig (Zeile 35), danach zusatztage = 1. T4: tage = 8 → zusatztage = 2.
Warum belegt dies die Ursache?	Laut Regel ist Tag 7 gratis und Tag 8 ist genau 1 Zusatztag. Der Debugger zeigt, dass bei 7 der Ja-Zweig läuft und bei 8 zwei Zusatztage gezählt werden. Bedingung und Abzug sind beide um 1 verschoben.
Gezielte Codekorrektur (erst jetzt!)	alt: if (tage >= 7) / zusatztage = tage - 6; neu: if (tage > 7) / zusatztage = tage - 7;
Direkter Nachtest: Soll / Ist	T3 Soll: 0 Tage, CHF 0.00 / Ist: 0 Tage, CHF 0.00 → bestanden T4 Soll: 1 Tag, CHF 0.50 / Ist: 1 Tag, CHF 0.50 → bestanden
PAP-Stelle, die dadurch angepasst werden muss	Raute «tage >= 7?» wird zu «tage > 7?»; Rechteck «zusatztage = tage - 6» wird zu «zusatztage = tage - 7»

Diagnosekarte 3

Was wird dokumentiert?	Dein Befund (VOR / NACH der Korrektur)
Meine Bezeichnung für die Ursache	Falscher Tagessatz (0.75 statt 0.50)
Test-ID und Eingabe	T5: Menü 1, Tage 12
Erwartetes Ergebnis (Soll)	Zusätzliche Tage 5, CHF 2.50
Beobachtetes Ergebnis (Ist)	Originalcode: Zusätzliche Tage: 6, CHF 4.50. Nach Korrektur von Karte 2: Zusätzliche Tage: 5, CHF 3.75
Vermutete Ursache / Hypothese	Hypothese B: In gebuehr = zusatztage * 0.75m stimmt der Preis pro Tag nicht mit der Regel (CHF 0.50) überein.
Breakpoint (Datei / Stelle)	Program.cs, Zeile 37, gebuehr = zusatztage * 0.75m;
Variablenwerte und ausgeführter Zweig	Vor F10: zusatztage = 5, gebuehr = 0. Nach F10: gebuehr = 3.75 → 3.75 / 5 = 0.75 pro Tag.
Warum belegt dies die Ursache?	zusatztage ist nach Karte 2 bereits richtig (5), trotzdem ist gebuehr falsch. Der Fehler kann also nur im Faktor liegen.
Gezielte Codekorrektur (erst jetzt!)	alt: gebuehr = zusatztage * 0.75m; neu: gebuehr = zusatztage * 0.50m;
Direkter Nachtest: Soll / Ist	Soll: CHF 2.50 / Ist: CHF 2.50 → bestanden
PAP-Stelle, die dadurch angepasst werden muss	Rechteck «gebuehr = zusatztage × 0.75» wird zu «gebuehr = zusatztage × 0.50»

Diagnosekarte 4

Was wird dokumentiert?	Dein Befund (VOR / NACH der Korrektur)
Meine Bezeichnung für die Ursache	Falsche Höchstgebühr (12 statt 10)
Test-ID und Eingabe	T6: Menü 1, Tage 40
Erwartetes Ergebnis (Soll)	CHF 10.00
Beobachtetes Ergebnis (Ist)	CHF 12.00 (auch im Originalcode und nach Korrektur von Karte 2 und 3)
Vermutete Ursache / Hypothese	Die Bedingung gebuehr > 10 ist richtig, aber im Ja-Zweig wird ein falscher Wert gesetzt.
Breakpoint (Datei / Stelle)	Program.cs, Zeile 38, if (gebuehr > 10m), danach F10 über Zeile 40
Variablenwerte und ausgeführter Zweig	gebuehr = 16.50 (33 × 0.50), Bedingung true → Ja-Zweig, nach Zeile 40 ist gebuehr = 12.
Warum belegt dies die Ursache?	Der richtige Zweig wird ausgeführt, aber der zugewiesene Wert 12 widerspricht der Regel «Höchstgebühr CHF 10.00».
Gezielte Codekorrektur (erst jetzt!)	alt: gebuehr = 12m; neu: gebuehr = 10m;
Direkter Nachtest: Soll / Ist	Soll: CHF 10.00 / Ist: CHF 10.00 → bestanden (Zusatztest 28 Tage: CHF 10.00)
PAP-Stelle, die dadurch angepasst werden muss	Rechteck «gebuehr = 12» wird zu «gebuehr = 10»

Falls sich eine Hypothese als falsch erweist, dokumentiere die widerlegende Debugger-Beobachtung. Auch das ist ein Lernnachweis.

Eigener Soll-PAP | Nach der Codekorrektur

4 Aus dem Ist-PAP selbst den Soll-PAP entwickeln

Schliesse die Diagnose ab und korrigiere nur tatsächlich nachgewiesene Fehlerstellen im Code.
Erstelle selbst eine Kopie von «Ist_PAP-Bibliotheksabrechnung.drawio»
Speichere die Kopie als «LS10_Soll_PAP_Nachname.drawio» ab.
Passe aufgrund deiner korrigierten Codezeilen die betroffenen Bedingungen, Berechnungen und Wege an.
Prüfe Notation, Ja-/Nein-Zweige, Menüschleife, Rücksprünge und Programmende.

Verfolge mindestens zwei vollständige Ausführungswege im Soll-PAP und vergleiche sie mit dem korrigierten Code.

Der Soll-PAP wird NICHT ausgeteilt: Er ist dein eigenständig bearbeitetes Ergebnis der Fehlersuche. Jede Änderung muss zu einer belegten Diagnose und Codekorrektur passen.

Geänderte PAP-Stelle	***Welche Diagnose belegt diese Änderung?***	***Was wurde im Code korrigiert?***
Raute «tage < 0?» → «tage <= 0?»	*Karte 1 (T1): bei tage = 0 lief der else-Zweig*	*Zeile 25: if (tage < 0) → if (tage <= 0)*
Raute «tage >= 7?» → «tage > 7?»	*Karte 2 (T3): bei tage = 7 lief der Ja-Zweig*	*Zeile 33: if (tage >= 7) → if (tage > 7)*
Rechteck «zusatztage = tage - 6» → «tage - 7»	*Karte 2 (T4): bei tage = 8 war zusatztage = 2 statt 1*	*Zeile 35: tage - 6 → tage - 7*
Rechteck «gebuehr = zusatztage × 0.75» → «× 0.50»	*Karte 3 (T5): 5 Zusatztage ergaben CHF 3.75*	*Zeile 37: 0.75m → 0.50m*
Rechteck «gebuehr = 12» → «gebuehr = 10»	*Karte 4 (T6): nach der Kappung war gebuehr = 12*	*Zeile 40: 12m → 10m*

Bei Bedarf weitere Zeilen hinzufügen.

5 Regression: Stimmen Regeln, Code und PAP überein?

Wiederhole **ALLE** selbst entworfenen Tests nach der Korrektur. Ergänze bei Bedarf neue Grenztests und mehrere Menüeingaben in einem Lauf.

Test-ID	Soll	Ist nach Korrektur	Bestanden?	PAP-Weg korrekt?
T1	Fehlermeldung, keine Gebühr	«Ungültige Ausleihdauer.»	Ja	Ja: tage <= 0? → Ja
T2	Fehlermeldung, keine Gebühr	«Ungültige Ausleihdauer.»	Ja	Ja: tage <= 0? → Ja
T3	CHF 0.00	Zusätzliche Tage: 0, CHF 0.00	Ja	Ja: tage > 7? → Nein
T4	CHF 0.50	Zusätzliche Tage: 1, CHF 0.50	Ja	Ja: tage > 7? → Ja, gebuehr > 10? → Nein
T5	CHF 2.50	Zusätzliche Tage: 5, CHF 2.50	Ja	Ja: tage > 7? → Ja, gebuehr > 10? → Nein
T6	CHF 10.00	Zusätzliche Tage: 33, CHF 10.00	Ja	Ja: gebuehr > 10? → Ja, gebuehr = 10
Menü	5 und -1: Hinweis + Menü; 1/8: CHF 0.50; 0: «Programm beendet.»	genau so in einem Lauf ausgegeben	Ja	Ja: auswahl == 1? → Nein → Hinweis → Rücksprung
Zusatz	1 Tag: CHF 0.00 / 27 Tage: CHF 10.00 / 28 Tage: CHF 10.00	CHF 0.00 / CHF 10.00 / CHF 10.00	Ja	Ja

PAP-Weg A:

Eingabe: auswahl = 1, tage = 12 → weiter == true? Ja → auswahl == 0? Nein → auswahl == 1? Ja → tage <= 0? Nein → tage > 7? Ja → zusatztage = 5 → gebuehr = 2.50 → gebuehr > 10? Nein → Ausgabe → Rücksprung zum Menü. Ergebnis: Zusätzliche Tage 5, CHF 2.50 (gleich wie im korrigierten Code).

PAP-Weg B:

Eingabe: auswahl = 1, tage = 0, danach auswahl = 0 → weiter == true? Ja → auswahl == 0? Nein → auswahl == 1? Ja → tage <= 0? Ja → «Ungültige Ausleihdauer.» → Rücksprung → auswahl == 0? Ja → weiter = false → weiter == true? Nein → «Programm beendet.» → Ende. Ergebnis: Fehlermeldung ohne Gebühr, dann Programmende (gleich wie im Code).

Belege und Abgabe

6 Zwei Debugger-Belege

Die Bilder müssen zwei unterschiedliche Beobachtungen zeigen: Haltepunkt, relevante Codezeile und Variablen. Formuliere jeweils eine fachliche Schlussfolgerung.

Datei	Test-ID / Diagnose	Was genau zeigt und beweist das Bild?
LS10_Debugger_1.png/jpg	T1 / Karte 1 (Prüfung Ausleihdauer)	Breakpoint in Zeile 25 if (tage < 0), Autos zeigt tage = 0. Nach F10 steht der gelbe Pfeil in Zeile 30 (else). Beweis: 0 wird nicht als ungültig erkannt, die Bedingung muss tage <= 0 lauten.
LS10_Debugger_2.png/jpg	T3 / Karte 2 (Grenze Tag 7)	Breakpoint in Zeile 33 if (tage >= 7), tage = 7. Nach F10 springt der Pfeil in Zeile 35 und zusatztage wird 1. Beweis: Tag 7 wird fälschlich als kostenpflichtiger Tag gezählt.

7 Reflexion

Welche deiner ursprünglichen Hypothesen war falsch oder musste geändert werden?

Hypothese A war nur teilweise richtig: Ich dachte, nur die Bedingung tage >= 7 ist falsch. Der Test mit 8 Tagen zeigte aber, dass auch tage - 6 einen Tag zu viel zählt. Den Fehler bei der Höchstgebühr (12 statt 10) hatte ich gar nicht vermutet, er fiel erst bei T6 auf.

Welche Debugger-Beobachtung half dir bei der schwierigsten Ursache?

Bei Karte 3 war im Debugger zusatztage bereits 5 (richtig), gebuehr aber 3.75. Dadurch war klar, dass der Fehler nicht mehr bei den Tagen, sondern beim Faktor 0.75 liegt. So konnte ich die Fehler von Karte 2 und 3 sauber trennen.

Woran erkennst du, dass dein eigener Soll-PAP zum korrigierten Code passt?

Jede geänderte Raute und jedes Rechteck im Soll-PAP entspricht genau einer korrigierten Codezeile (Zeilen 25, 33, 35, 37, 40). Ich habe Weg A (12 Tage) und Weg B (0 Tage, dann Beenden) im PAP verfolgt und mit dem Programm verglichen: gleiche Entscheidungen, gleiches Ergebnis.

Falls du KI verwendet hast: Wozu und wie hast du die Aussagen selbst geprüft?

Ich habe KI (Claude) verwendet, um Testfälle, Diagnosekarten und Formulierungen auszuarbeiten, aufgaben habe ich vorher selbst gelöst, nur die formulierung am schluss wurde mit Claude gemacht. Geprüft habe ich die Aussagen, indem ich alle Testfälle mit dem Original- und dem korrigierten Programm ausgeführt und die Ausgaben mit den vorher von Hand berechneten Sollwerten verglichen habe.

8 Git-Nachweis

Commit-ID (kurz)	Aussagekräftige Commit-Nachricht
2bdac8d	**LS10 Debugging-Labor abgeschlossenerstellt**

9 Checkliste für Microsoft Teams

LS10_Diagnose_Nachname.pdf - dieses **ausgefüllte** Protokoll mit eigenem Testplan, Diagnosekarten, PAP-Begründung, Regression und Reflexion.

LS10_Debugger_1.png/jpg und **LS10_Debugger_2.png/jpg** - **zwei unterschiedliche, begründete** Debugger-Belege.

LS10_Soll_PAP_Nachname.drawio - selbst erstellter, editierbarer PAP.

BibliothekDebug_Nachname.zip vollständig korrigiertes Visual-Studio-Projekt ohne «bin» und «obj» Ordner.

Git-Commit-ID und **Commit-Nachricht** sind oben in Punkt 8 eingetragen und alle oben genannten Abgaben vollständig im Remote Git-Repository eingecheckt.

Abgabetermin: gemäss Teams. Optional-Aufgaben zählen nicht zur Pflichtzeit.